Engine Showcase

See All 8 Engines in Action - Before You Choose

VIDEO GAME CREATION

A preview for families, ahead of Lesson 1

INSTRUCTOR NOTE - delete this box before sending to families. This document has 24 screenshot placeholders (3 per engine x 8 engines), each with an exact filename and a description of what to capture. Whenever convenient - pre-building the Lesson 1 Live Engine Tour projects is a natural time, since you'll have all 8 engines open anyway - capture each shot and drop it in the Screenshots_Inbox folder using the exact filename shown in its placeholder box. No need to crop or clean anything up first. Once a batch is in the inbox, they get annotated (circles/arrows/labels as needed) and dropped into the matching placeholder below, replacing the box.

How to Use This Document

This is a preview, not a tutorial. Every engine below is doing the exact same tiny thing: pressing a key causes a character to react. That's the very first hands-on exercise of the class (Lesson 2's "Key Press Reactor"), and it's a genuinely fair way to compare engines side by side, since it's the same problem solved eight different ways.

For each engine you'll see: what you're looking at, a shot of the project set up and ready, a shot of the actual code/blocks/nodes that make it work, and a shot of the result after pressing a key. Skim for whichever engines look interesting, then check the Engine & Platform Guide for the fuller picture on cost, hardware needs, and install steps before making a final choice.

Coding styles are tagged for each engine - Block (drag-and-snap puzzle pieces), Visual (connecting events/nodes without typing), or Scripting (typed text code). You are not locked into one style for the whole year: several of these engines let you use more than one style in the same project, and the class deliberately rotates through all three all year, so trying one now doesn't close any doors.

The Engines

Scratch

- **Coding style:** Block

Press the space bar and your sprite flashes a different color for a moment - the simplest possible "press a key, something happens" reaction, built from two snapped-together puzzle-piece blocks.

> **SCREENSHOT NEEDED: `showcase_scratch_setup.png`**
> Scratch editor open, placeholder sprite (e.g. the blue circle character) added and selected in the Sprite panel.
>
> **SCREENSHOT NEEDED: `showcase_scratch_code.png`**
> The two connected blocks in the code area: "when [space] key pressed" on top of "change color effect by 25."
>
> **SCREENSHOT NEEDED: `showcase_scratch_result.png`**
> The stage right after pressing space - sprite mid color-change.

Godot Engine

- **Coding style:** Scripting

Hold the Right arrow key and the character slides across the screen - a few lines of GDScript checked every frame.

> **SCREENSHOT NEEDED: `showcase_godot_setup.png`**
> Godot editor with a Sprite2D node in the scene tree, placeholder image assigned as its Texture in the Inspector.
>
> **SCREENSHOT NEEDED: `showcase_godot_code.png`**
> The attached script showing the _process(delta) function with the "if Input.is_action_pressed" check.
>
> **SCREENSHOT NEEDED: `showcase_godot_result.png`**
> The running scene (F6) with the sprite partway across the screen after holding Right.

Unity

- **Coding style:** Scripting

Press Play, hold an arrow key or A/D, and a placeholder GameObject slides across the Scene view - Unity's C# Update() loop in action.

> **SCREENSHOT NEEDED: `showcase_unity_setup.png`**
> Unity Hierarchy panel showing the Player GameObject with a Sprite Renderer component, placeholder image assigned.

SCREENSHOT NEEDED: `showcase_unity_code.png`
The PlayerMove script open in the editor, showing the Update() method checking Keyboard.current.rightArrowKey.isPressed.

SCREENSHOT NEEDED: `showcase_unity_result.png`
Play mode running, Player object visibly moved right in the Game view.

GameMaker

- **Coding style:** Visual or Scripting (shown here in GML text)

Hold the Right arrow key in a running room and the object steps to the right, one Step-event line of GML at a time - or the same logic built from GML Visual blocks instead.

SCREENSHOT NEEDED: `showcase_gamemaker_setup.png`
GameMaker workspace with obj_player in the Room Editor, spr_player (placeholder image) assigned as its sprite.

SCREENSHOT NEEDED: `showcase_gamemaker_code.png`
The Step event showing "if (keyboard_check(vk_right)) x += 4;" (or the equivalent GML Visual blocks).

SCREENSHOT NEEDED: `showcase_gamemaker_result.png`
The running game (F5) with the object visibly moved right after holding the key.

GDevelop

- **Coding style:** Visual

Click Preview and all 4 arrow keys already move the Player object around - GDevelop's built-in "Top-down movement" behavior does the work with zero hand-written events.

SCREENSHOT NEEDED: `showcase_gdevelop_setup.png`
GDevelop scene editor with the Player object selected and the Behaviors tab open, "Top-down movement" behavior listed as added.

SCREENSHOT NEEDED: `showcase_gdevelop_code.png`
The (mostly empty) event sheet, showing that no manual events are needed for basic movement - or the manual "Key pressed" event if that method was used instead.

SCREENSHOT NEEDED: `showcase_gdevelop_result.png`
Preview mode running with the Player object visibly moved after arrow key presses.

TIC-80

- **Coding style:** Scripting

Hold the Right arrow key in this tiny fantasy console and a hand-drawn 8x8 sprite steps across the 240x136 screen - six lines of Lua, typed directly into the built-in code editor.

SCREENSHOT NEEDED: `showcase_tic80_setup.png`
TIC-80's Sprite Editor with a simple character drawn on sprite index 1.

SCREENSHOT NEEDED: `showcase_tic80_code.png`
The built-in code editor showing the short TIC() function with "if btn(3) then x=x+1 end."

SCREENSHOT NEEDED: `showcase_tic80_result.png`
The running cart (after typing "run") with the sprite moved right on screen.

001 Game Creator (Windows only)

- **Coding style:** Visual

Press the Right arrow key and "main," the default player Actor, steps to the right - built entirely from an Input Trigger and a Position Actor event, no code typed at all.

SCREENSHOT NEEDED: `showcase_001gc_setup.png`
001 Game Creator's default Map with the "main" Actor selected, placeholder sprite optionally assigned in its Body settings.

SCREENSHOT NEEDED: `showcase_001gc_code.png`
The Input Triggers window showing the Right Arrow "Pressed" trigger, and its attached Script Editor graph with a single Position Actor event.

SCREENSHOT NEEDED: `showcase_001gc_result.png`
Play/Test mode with "main" visibly moved right after the key press.

Unreal Engine (optional advanced track)

- **Coding style:** Visual (Blueprints)

Press Play, tap the Right arrow key, and a placeholder Cube slides across the level - an Enhanced Input Action Event node (IA_MoveRight, mapped to the Right arrow in an Input Mapping Context) wired to an Add Actor World Offset node in the Level Blueprint. The heaviest engine on our list, so this one's for students with capable hardware.

SCREENSHOT NEEDED: `showcase_unreal_setup.png`
The level viewport with a Cube Actor placed (from the Place Actors panel), selected in the World Outliner.

SCREENSHOT NEEDED: `showcase_unreal_code.png`
The Level Blueprint graph showing the Enhanced Input Action Event (IA_MoveRight) node wired into an Add Actor World Offset node.

SCREENSHOT NEEDED: `showcase_unreal_result.png`
Play-in-editor with the Cube visibly moved after the key press.

Which One Should I Try First?

If you're not sure yet, the Engine & Platform Guide's "Which Engine Fits My Laptop?" table is the fastest filter - hardware rules out options quickly, and everything left after that is a matter of taste. A few rules of thumb:

- Never coded before, or want the fastest path to something playable? Start with Scratch.
- Oldest laptop in the house, or want a zero-install option? GDevelop runs entirely in a browser.
- Want a real scripting language without a heavy install? Godot is the class's recommended default.
- Aiming at an industry-recognized skill and certification path? Unity.
- On Windows with a very old or low-power laptop? 001 Game Creator is the lightest option on this list.
- Have a newer, powerful laptop and want the most advanced 3D toolset? Unreal Engine (optional track).

Whatever you pick, you can change your mind later - every lesson keeps working examples across all 8 engines all year.